

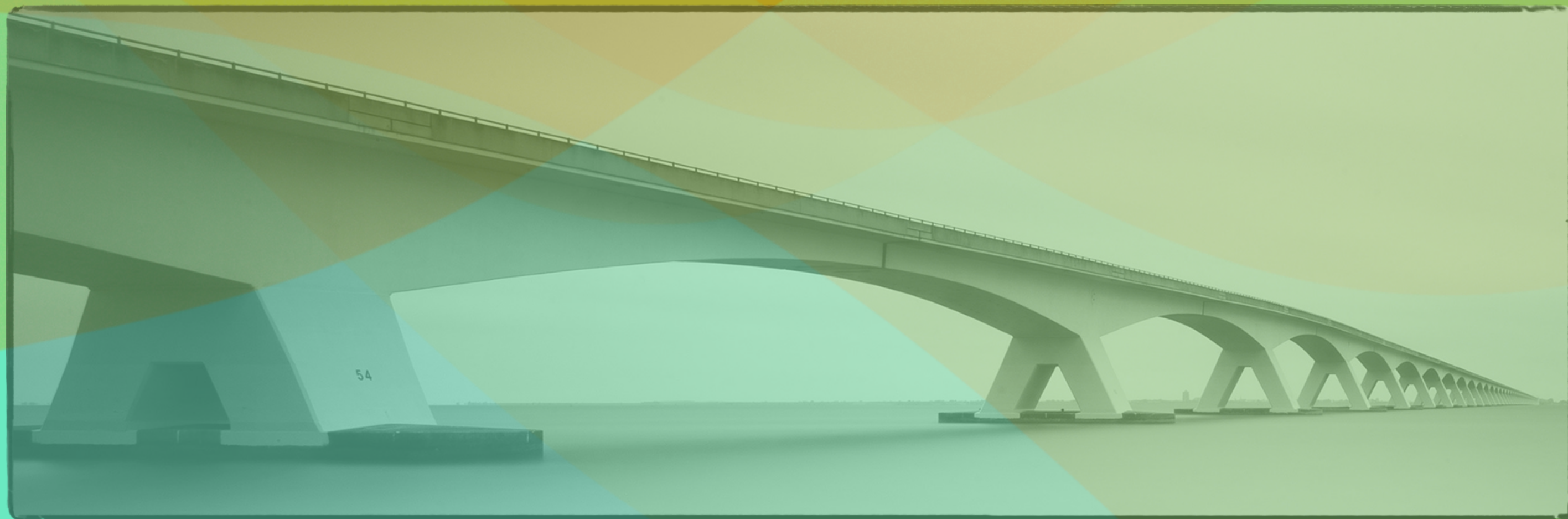
Python & Golang

Secure Communication

HTTPs - Encryption & Authentication
In 5 steps !

Step 0: Preparation

- ❖ Ensure Python3 is installed
- ❖ Create a python virtual environment
 - ❖ Install gunicorn and flask
- ❖ Install golang
- ❖ Install OpenSSL



Step 1:

Create the Certificates

❖ CA :
`openssl genrsa -out ca.key 2048`

`openssl req -new -key ca.key -x509 -days 3650 -out ca.crt -subj /C=US/ST=NC/O="Localhost Ltd"/CN="Localhost Root"`

❖ SERVER :
`openssl genrsa -out server.key 2048`

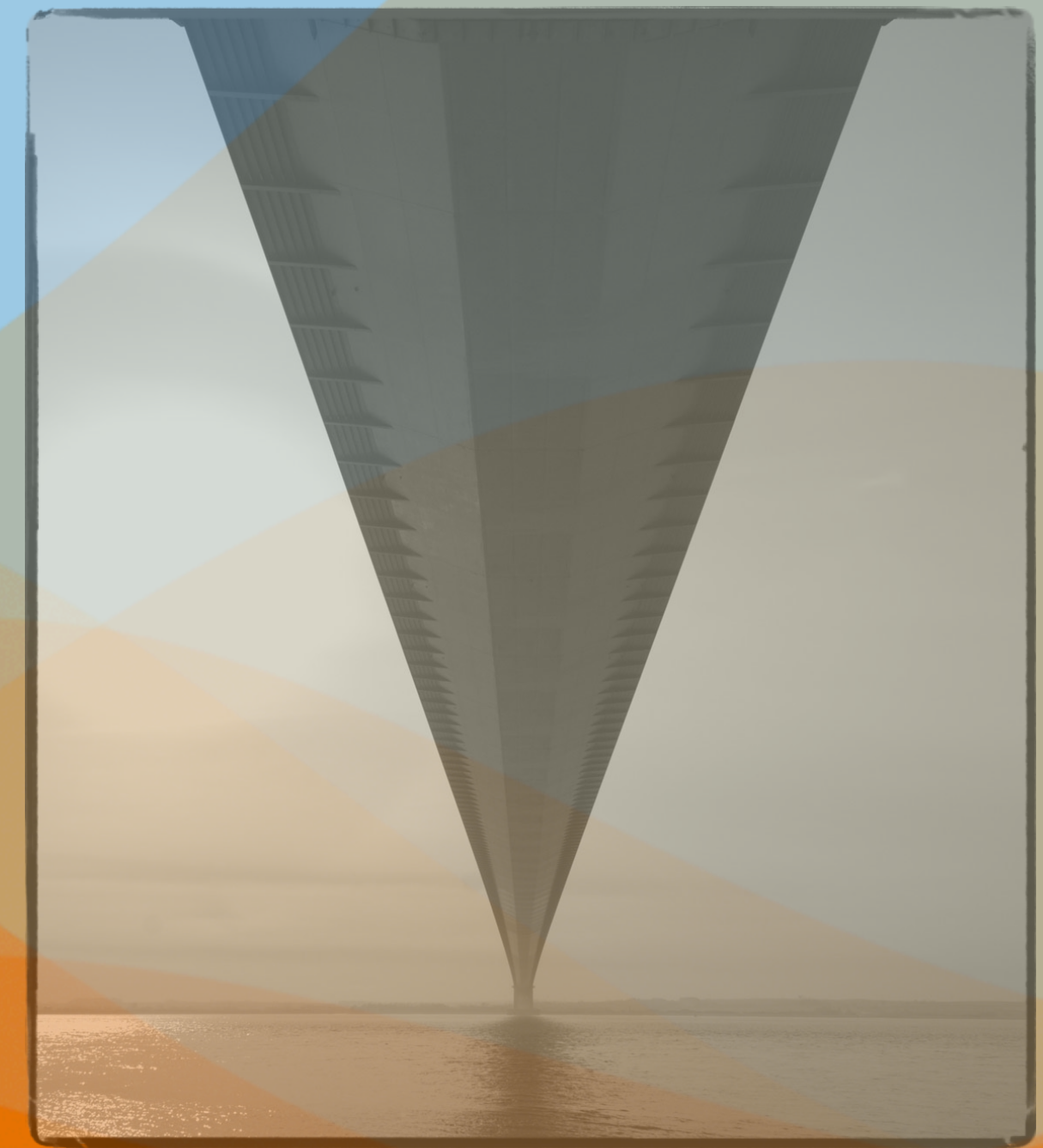
`openssl req -new -nodes -key server.key -out server.csr -subj /C=US/ST=NC/L=RTP/O="Localhost Server"/CN=localhost`

`openssl x509 -req -extfile <(printf "subjectAltName=DNS:localhost,DNS:www.sridhar.com") -days 365 -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt`

❖ CLIENT :
`openssl genrsa -out client.key 2048`

`openssl req -new -nodes -key client.key -out client.csr -subj /C=US/ST=NC/L=RTP/O="Localhost Client"/CN=localhost`

`openssl x509 -req -extfile <(printf "subjectAltName=DNS:localhost,DNS:www.sridhar.com") -days 365 -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.crt`



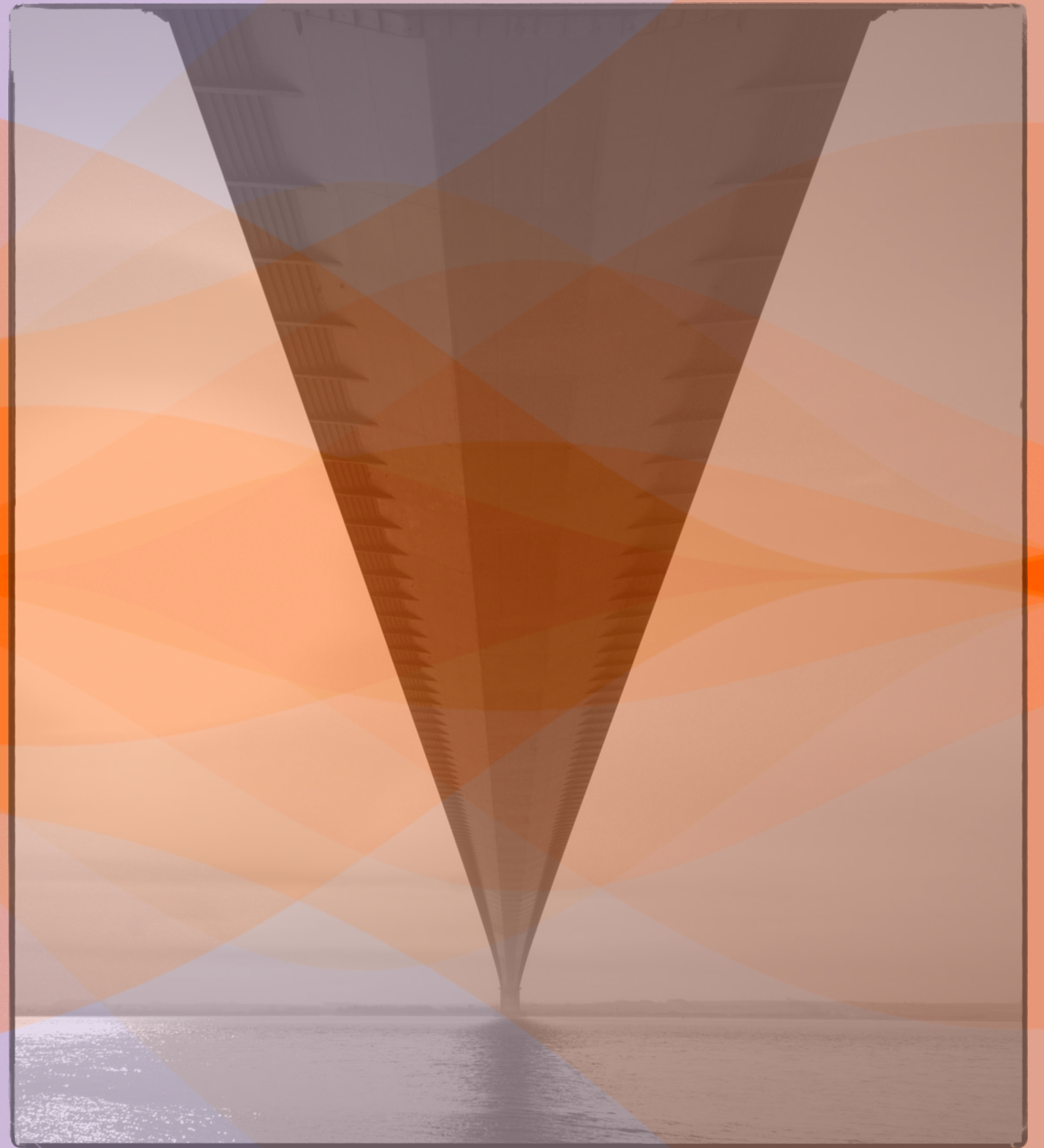
Step 2:

Python Server

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route('/')  
def index():  
    return "Hello from Flask!"
```



Step 3:

Launch Python Server with gunicorn & Certificates

```
gunicorn flaskserver:app --bind localhost:4433 --keyfile golang/server.key --certfile golang/server.crt --cert-reqs 2 --ca-certs golang/ca.crt
```



Golang Client with Certificates

Step 4:

```
package main

import (
    "crypto/tls"
    "crypto/x509"
    "flag"
    "io/ioutil"
    "log"
    "net/http"
)

func createClientConfig(ca, crt, key string) (*tls.Config, error) {
    caCertPEM, err := ioutil.ReadFile(ca)
    if err != nil {
        return nil, err
    }

    roots := x509.NewCertPool()
    ok := roots.AppendCertsFromPEM(caCertPEM)
    if !ok {
        panic("failed to parse root certificate")
    }

    cert, err := tls.LoadX509KeyPair(crt, key)
    if err != nil {
        return nil, err
    }
    return &tls.Config{
        Certificates: []tls.Certificate{cert},
        RootCAs:      roots,
    }, nil
}
```

```
func main() {
    ca := flag.String("ca", "./ca.crt", "root certificate")
    crt := flag.String("crt", "./client.crt", "certificate")
    key := flag.String("key", "./client.key", "key")
    flag.Parse()

    config, err := createClientConfig(*ca, *crt, *key)
    if err != nil {
        log.Fatal("config failed: %s", err.Error())
    }

    client := &http.Client {
        Transport: &http.Transport {
            TLSClientConfig: config,
        },
    }

    resp, err := client.Get("https://localhost:4433")
    if err != nil {
        panic(err)
    }

    respBody, _ := ioutil.ReadAll(resp.Body)
    log.Printf(string(respBody))
}
```





Step 5:

Run The Client

```
# go run client.go  
2024/01/29 08:06:36 Hello from Flask!  
#
```